Malware Detection and Analysis Using LLMs: A Review of Emerging Trends and Techniques

Mashel Albalooshi
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
24002232@student.buid.ac.ae

Peter Chang
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
24000753@student.buid.ac.ae

Suleiman Y. Yerima
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
suleiman.yerima@buid.ac.ae

Thekra Altamimi
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
24002174@student.buid.ac.ae

Hamda AlFalasi
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
24002186@student.buid.ac.ae

Abdulrahman Almarzooqi
Faculty of Engineering and IT
The British University in Dubai
Dubai, United Arab Emirates
24000119@student.buid.ac.ae

2025 2nd International Conference on Artificial Intelligence, Metaverse, and Cybersecurity (ICAMAC) | 979-8-3315-7225-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICAMAC67779.2025.11398140

***Abstract*—The increasing sophistication of malware characterized by polymorphism, obfuscation, and adversarial behaviors poses a significant challenge to traditional detection methods. As cyber threats outpace conventional signature-based and heuristic techniques, Large Language Models (LLMs) have emerged as a promising solution in advancing malware detection and analysis. This systematic review consolidates findings from 10 recent studies published between 2021 and 2025 that explore the application of LLMs in malware detection across static, dynamic, and hybrid analysis environments. LLMs such as GPT-4o, BERT, Gemini and Mistral 7B have demonstrated remarkable capabilities in semantic code understanding, behavioral pattern recognition, and zero-shot threat inference. These models enable automated extraction of malware capabilities, identification of Indicators of Compromise (IoCs), and generation of human-readable threat summaries thus enhancing analyst efficiency and detection accuracy. Frameworks like LLM-MalDetect and GENTTP showcase how prompt engineering, string feature analysis, and transformer-based embeddings can be harnessed to detect evasive and novel malware. Despite their potential, the deployment of LLMs in operational settings remains constrained by challenges including explainability, latency, model size, and dual-use risks. This review identifies current gaps in benchmarking, generalization, and ethical governance, while outlining future directions involving multimodal data fusion, lightweight deployment, and integration with real-time security operations. It is evident from our research that LLMs are reshaping malware defense strategies, offering a scalable, context-aware, and semantically enriched approach to modern threat detection.**

***Index Terms*—- Large Language Model; LLM(s); Artificial Intelligence; Malware detection; Malware analysis; static analysis; dynamic analysis; Indicator of Compromise; TTP**

I. Introduction

The rapid evolution of cyber threats has outpaced conventional malware detection techniques, necessitating the integration of advanced artificial intelligence (AI) methods into cybersecurity defense systems [1]. Among these, Large Language Models (LLMs), a class of deep learning models pre-trained on massive textual corpora have emerged as a transformative force in the field of malware detection and behavioral analysis. Originally designed for natural language processing (NLP) tasks, LLMs like GPT-4, BERT, Gemini, Claude, and Mistral 7B have demonstrated superior capabilities far beyond linguistic applications, including code analysis, anomaly detection, semantic analysis and even zero-shot threat interpretation [2] [3]. As malware becomes increasingly polymorphic, evasive, and capable of leveraging AI for offensive purposes, the need for semantically aware, context-sensitive detection mechanisms has never been more pressing. This review synthesizes the most recent and impactful research, spanning 2021 to 2025, that explore the integration of LLMs into various facets of malware detection.

Traditional static and dynamic malware detection methods rely heavily on signature matching, rule-based heuristics, and handcrafted feature engineering [4]. While effective in known threat scenarios, these approaches falter in the face of zero-day exploits, packing techniques, obfuscated code, and metamorphic malware. LLMs, by contrast, offer a paradigm shift by leveraging their ability to capture semantic relationships, understand complex execution flows, and generalize across syntactically diverse code samples. For instance, frameworks like LLM-MalDetect use prompt-engineered inputs to classify Android malware based on permissions and API sequences, achieving state-of-the-art accuracy across multiple datasets (e.g., Drebin, AndroZoo). Similarly, tools such as GENTTP apply zero-shot LLM reasoning to extract Tactics, Techniques, and Procedures (TTPs) from open-source malware packages, demonstrating the feasibility of training-free behavioral threat modeling [5] [6].

The evolving sophistication of malware, driven by polymorphism, obfuscation, and evasive execution tactics, continues

979-8-3315-7225-9/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

to outpace traditional detection approaches. With the advent of Large Language Model (LLM)-based malware generation platforms such as WormGPT and FraudGPT, cybercriminals now have unprecedented access to tools that can automate and enhance malicious code creation, phishing content, and social engineering tactics [4] [7]. This rising offensive use of generative AI underscores the urgent need for equally advanced LLM-assisted malware detection and analysis systems [2] [3] [4]. Despite their promise, LLMs in malware detection are not without challenges. Studies consistently report limitations around explainability, inference latency, and token limits, especially when handling long and complex binary sequences. The black-box nature of transformer models raises concerns around trust and transparency, particularly in regulated environments like finance and critical infrastructure [4]. Moreover, dual-use risks where LLMs are repurposed to generate polymorphic or evasive malware underscore the ethical and security implications of unrestricted access to generative AI [7]. Current research also indicates a lack of standardized benchmarks, fragmented datasets, and over-reliance on specific environments (e.g., Cuckoo Sandbox, outdated datasets), which hinder reproducibility and model generalization [4] [7]. To address these gaps, researchers propose several future directions: from domain-specific fine-tuning and federated deployment to multimodal fusion of static, dynamic, and textual features. Explainable AI (XAI) techniques, such as SHAP and attention-based heatmaps, are also being explored to interpret LLM predictions and reduce false positives. Integrations with SIEM, EDR, and MLOps pipelines are envisioned to bring LLMs into real-time detection and response workflows [8].

In summary, LLMs represent a critical evolution in the malware detection landscape, offering contextual, adaptive, and language-agnostic analysis that is unattainable through traditional approaches. This review paper presents a structured synthesis of 10 cutting-edge studies, drawing insights into the architectural designs, detection capabilities, limitations, and prospective advancements in LLM-driven malware detection. As threats continue to escalate in sophistication and scale, understanding and harnessing the full potential of LLMs will be pivotal to fortifying modern cybersecurity ecosystems.

II. Background

A. Traditional Malware Detection Techniques and Their Limitations

For decades, cybersecurity defenses have relied on traditional malware detection techniques such as static and dynamic analysis to identify and mitigate malicious software. These approaches, while effective in earlier threat environments, are increasingly being outmaneuvered by modern, evasive malware. As threat actors now exploit artificial intelligence including Large Language Models (LLMs) to create polymorphic and highly targeted malware, the limitations of conventional techniques have become more pronounced [3].

Static analysis involves examining a binary file without executing it. This method extracts low-level features such as opcode sequences, file metadata, import tables, and header structures. Tools like IDA Pro and Python libraries such as PEfile are commonly used to derive signature-based or heuristic indicators from Portable Executable (PE) files. Although efficient and fast, static analysis fails to detect obfuscated or encrypted payloads. Techniques such as code packing, metamorphism, and polymorphism are specifically designed to evade these signature-based detection mechanisms [3] [4] [7].

Dynamic analysis, on the other hand, executes malware in controlled environments such as sandboxes (e.g., Cuckoo) or uses Dynamic Binary Instrumentation (DBI) tools like Intel Pin to observe runtime behaviors API calls, system modifications, and network activity. While more robust against obfuscation, dynamic analysis is increasingly defeated by anti-analysis strategies. Modern malware can detect sandbox environments or instrumentation frameworks and adapt its behavior to appear benign or cease execution altogether [3] [4] [7]. One of the biggest challenges of Malware analysis and detection was the velocity, variety and volume of the daily occurrence of new malwares in the wild. Recent statistics from AV-Test.org show that there have been more than 3.5 million new malwares discovered in just the last 14 days at the time of writing (as of June 2025) as shown in Figure 1 [9]. This wave of increasing amounts of new malware also aligns with the recent explosion of LLMs in the market lowering the entry point to malware authors with the assistance of powerful LLMs [6].

Fig. 1. New Malware 2025 Stats [9]

B. The Emergence of LLMs in Malware Detection and Analysis

The advent of Large Language Models (LLMs) has redefined what is possible in artificial intelligence. Initially developed for natural language processing tasks, LLMs such as BERT, GPT-4, CodeT5, and Mistral 7B have demonstrated remarkable generalization capabilities across diverse domains including cybersecurity [10]. Their ability to learn syntactic and semantic patterns, model code behavior, and adapt to new inputs without task-specific training makes them ideally suited to the complex, evolving problem of malware detection [8].

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

The development of LLMs traces back to the evolution of transformer architectures in 2017, which enabled parallelized attention mechanisms capable of capturing long-range dependencies in data. With billions of parameters trained on massive corpora of text and code, modern LLMs can perform reasoning, generate code, interpret behavior, and even classify unseen inputs using zero-shot or few-shot learning. This makes them well-equipped to detect previously unknown malware variants that exhibit new or obfuscated behaviors [10]. Recent research demonstrates how LLMs can be embedded into malware detection workflows. For example, LLM-MalDetect leverages Mistral 7B fine-tuned on structured APK features and permission strings to identify Android malware with over 98% accuracy. Similarly, GENTTP uses GPT-4 for zero-shot extraction of Tactics, Techniques, and Procedures (TTPs) directly from malware packages without requiring labelled training data. Reverse engineering tools like Gemini Pro are also used to generate natural language summaries and Indicators of Compromise (IoCs) from disassembled binaries, improving analyst productivity and detection accuracy [10] [5] [6].

LLMs also offer new possibilities in semantic analysis, allowing for the transformation of opcode or API call sequences into high-level behavioral descriptors. This contextual reasoning is invaluable for detecting sophisticated malware, particularly those leveraging AI for concealment or conditional payload delivery. Furthermore, by integrating with Security Information and Event Management (SIEM) systems and MLOps pipelines, LLMs can contribute to real-time incident response and continuous learning models. In an age where attackers are empowered by generative AI, defenders must respond with equally intelligent and adaptive tools. The integration of LLMs into cybersecurity workflows marks a critical evolution, positioning them not only as detection engines, but as essential components in the next generation of resilient, explainable, and proactive malware defense systems.

III. Methodology

A. Objectives

The primary objective of this systematic literature review is to critically examine and synthesize existing research on the application of Large Language Models (LLMs) for malware detection and analysis. Over the last five years, LLMs such as GPT-4, BERT, CodeT5, and Mistral 7B etc. have demonstrated unprecedented potential in semantically interpreting code, classifying malware behaviors, and automating reverse engineering tasks. As cyber threats grow more evasive—through polymorphic malware, LLM-powered phishing, and modular payloads, traditional signature and heuristic-based detection methods are increasingly insufficient. This review seeks to explore how LLMs can bridge these gaps by offering context-aware, zero-shot or few-shot detection capabilities and integrating with security operations for real-time threat intelligence. Through a structured analysis of ten peer-reviewed and preprint publications from 2021 to 2025, this study aims to identify the current capabilities, architectural trends, limitations, and emerging use cases of LLMs in malware detection. The review will also evaluate the challenges faced in practical deployment, such as computational overhead, explainability, token limitations, and dual-use risks. Ultimately, this SLR will provide a foundation for guiding future research directions and informing the development of AI-enhanced security systems that are both resilient and adaptive to evolving malware ecosystems.

B. Research Questions

To guide the analysis and extraction of insights from the reviewed literature, this study is structured around the following research questions:

RQ1: What techniques and architectures have been proposed or implemented to integrate LLMs into malware detection and analysis workflows? This question seeks to understand the diverse approaches, such as prompt engineering, fine-tuning, and embedding transformation—used to embed LLMs into static and dynamic malware analysis pipelines.

RQ2: What are the key strengths and limitations of LLMs compared to traditional and deep learning-based malware detection methods? This question explores the unique capabilities of LLMs in handling obfuscated code, extracting behavioral semantics, and enabling zero-shot classification, while also evaluating their challenges related to latency, explainability, and model robustness.

RQ3: What future directions, risks, and ethical considerations must be addressed to safely and effectively deploy LLM-based malware detection systems at scale? This question aims to surface gaps in current research, including concerns around dual use (e.g., WormGPT, and FraudGPT), dataset availability, adversarial vulnerabilities, and regulatory compliance, and propose future research trajectories.

C. Paper Selection

Primary studies were selected by applying PRISMA methodology where pre-defined keywords to online reputable scientific libraries. The search term “Malware analysis”, “LLMs”, “Large Language Models”, “Malware detection” were used to find relevant scientific studies from IEEE (Institute of Electrical and Electronics Engineering), ScienceDirect, ResearchGate, ACM and arXiv.org digital libraries. Due to the limited scope of this literature review and limited access to other online libraries, most of the initially selected studies are from ScienceDirect, IEEE and arXiv.org. The inclusion criteria are listed below:

- The selected papers should focus on how LLMs are applied to assist Malware detection and analysis.
- The paper’s publication date should be within the previous 5 years (2021-2025) focusing on most recent studies.
- All papers considered are journal articles, research papers and preprints from reputable online repositories.
- The selected papers must be published in English language.
- Duplicated papers will be excluded.

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

- Further full-text paper assessment conducted on selected papers to include the most relevant papers for this review

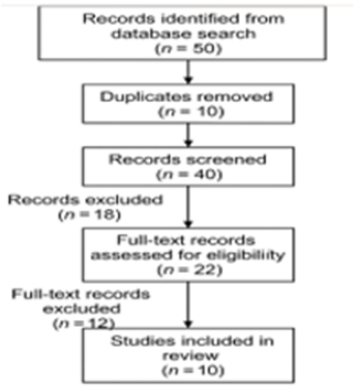

Fig. 2. Paper Selection Process

Figure 2 shows the steps followed in the studies selection for this review. After an initial database search using the previously mentioned keywords, due to the time constraints only 50 papers were identified with 10 of them being duplicates. After an examination of the titles/abstract/introduction of each paper, it was determined that 22 of them meet the criteria for our review. Following a full text examination, a further 12 papers were excluded, and only 10 papers were selected as they meet our selection criteria for this review.

IV. A Review of Recent Works Leveraging LLMs for Malware Detection and Analysis

Some malware, such as spywares or trojan horses, would utilize keylogger and screen capture functions to secretly log the victim's keystrokes on the infected devices and send them to the attacker for future attacks. Andrei Mogagea presented a novel approach that combines Large Language Models (LLMs) with a taint-based binary malware analysis framework to detect and understand specific malware capabilities, such as keylogging and screen capturing [10]. This approach combines dynamic taint analysis, temporal logic and binary instrumentation to detect if malware possesses specific capabilities. It integrates Mistral 7B to aid in malware analysis such as discovering new capabilities, identifying relevant Windows APIs, Generating C++ snippets as references and creating more creative and flexible detection rules. LLMs has shown superior capabilities to expand the scope and diversity of possible implementations of malware behavior, helping analysts encode more comprehensive capability rules for better detection accuracy especially in suggesting API chains, coding structures and low-level behavior logic used in malware which improves rules generation speed and accuracy [10]. This paper highlights a powerful shift from static rule-based detection to AI-augmented malware understanding. Rather than replacing analysts, LLMs could empower cybersecurity researchers to think more broadly and systematize the detection of evolving threats if a Human-Centric approach is utilized. Some limitations are also pointed out by the author such as inherited issues with LLMs where outputs were sometimes inaccurate or incomplete which requires expert validation and tuning due to lack of contextual precision. Future research could focus on making LLMs aware of the rule-defining logic to autonomously generate capability rules based on malware sample behavior.

Android's open nature and widespread use make it a prime target for malware. With threats like banking trojans, spyware, and ransomware evolving, users face increasing risks of financial loss, data theft, and privacy invasion. Enhanced detection is crucial to protect personal and corporate information on these ubiquitous devices. Feng et al. proposed a novel Android malware detection framework called LLM-MalDetect leveraging LLMs (Mistral 7B fine-tuned via LoRA to reduce GPU memory footprint while achieving high performance) to overcome the limitations of static feature-based methods by integrating string-based semantic analysis, prompt engineering, and fine-grained feature selection [5]. LLM-MalDetect integrated LLM's powerful semantic understanding ability to interpret permission declarations, API usage and app strings to enhance malware feature representation. Specially crafted prompts were used to bypass model hallucination improving model accuracy. LLM's self-attention mechanism was used to dynamically model local and global semantic dependencies between features to greatly enhance the understanding of the malware intricacies. The experiment results indicate that LLM-MalDetect outperformed state-of-the-art baselines across five datasets (Drebin, CICAndMal, AndroZoo-2023, AndroZoo-2024 and RePack) with high detection rate, accuracy and F1 Scores [5]. Even though the evaluation of the model using labelled public datasets has shown promising results, there are still a few limitations in this model.

Latency is a main challenge due to the limited hardware resources on mobile devices to take advantage of powerful LLM features so future work related to on-device lightweight LLMs could be explored for client-side integration and edge computing scenarios with smaller transformer models. Overall, LLM-MalDetect demonstrates a powerful new direction in malware detection blending feature engineering with contextual reasoning of LLMs. It achieved state-of-the-art accuracy with strong generalization across diverse datasets while maintaining robustness against obfuscation and repackaging. The continuous development of more powerful computing resources on mobile devices plus more efficient and smaller LLMs demonstrate that this framework has a huge potential to evolve into a powerful force in malware detection on mobile devices.

In 2024, Wong et al. [11] asked a blunt question: can GPT-4 do a malware analyst's grunt work faster than symbolic execution? The study tests the model against 56 "logic bombs" that represent real-world evasive behaviors, including unbounded loops, covert data propagation, and environment-based checks. GPT-4 discovered the trigger logic in most cases and provided plain English rationales, while symbolic analysis times-out or

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

drowns in path-explosion. The model was also effective when applied to real malware samples, successfully identifying evasion triggers such as keyboard layout checks and specific file path dependencies. However, limitations include occasional inaccuracies in pseudocode generation and reasoning errors, often stemming from differences in language semantics or training data overlap. Some logic bombs may also exist in the model's pretraining corpus, raising concerns about test validity. The authors propose future work including testing on unseen malware samples, integrating LLMs with debuggers and toolchains, and conducting user studies to measure real-world malware analyst benefit. Overall, the study in [11] shows that LLMs can be powerful assistants in malware analysis, not only outperforming legacy tools in specific contexts but also enhancing human decision-making through transparent reasoning.

The study conducted by Zhao et al. [12] was focused on trying to improve malware detection on Android devices by integrating it with AppPoet, an LLM model that has the functionality of multi-view prompt engineering. It works by extracting application features such as permissions and APIs and organizing them in structured ways that is easy to interpret. The AppPoet LLM uses its prompt engineering capability to craft prompts to assist in generating semantic summaries and function descriptors. A neural network is then used to classify and identify malware. The LLM also has the ability to generate human-readable reports. This method can be considered an improvement over the traditional methods of utilizing strings and images for detection.

The limitations observed with the model in [12] include being prone to LLM hallucinations when the input is unclear and the high cost associated with acquiring this model in a large environment. To further progress with this model in the future, the prompts need to be refined along with looking into ways to reduce the cost of the model and make it more efficient for deployment in large environments.

The paper by Sanchez et al. [13] explores the effectiveness of various Large Language Models (LLMs) for malware detection using system call sequences, comparing six models including BERT, GPT-2, BigBird, Longformer, and Mistral on over 1TB of syscall data from a Raspberry Pi-based setup. It introduces a pre-processing pipeline for tokenizing syscalls and shows that models with longer context windows (BigBird, Longformer at 4096 tokens) perform best, achieving approximately 0.86 accuracy, while larger-context models like Mistral under-perform due to overfitting. The study highlights the importance of context size in capturing behavioral patterns and recommends future work on anomaly detection, edge deployment via model quantization, and integrating temporal and network signals for real-time threat detection.

Williamson et al. [14] investigated how well Gemini Pro interprets the findings of reverse engineering on 10 different kinds of malware. By contrasting it with more conventional malware analysis techniques, the study seeks to evaluate the output produced by Gemini Pro in terms of readability and comprehensibility. The "Microsoft Malware Classification Challenge (BIG 2015) Dataset," which was made public by Microsoft in 2015 and comprises more than 20,000 malware samples from nine distinct families, served as the dataset for this investigation. The study reveals Gemini Pro's strong capability in analyzing reverse-engineered malware, offering clear, accurate insights into malware behavior. This highlights its potential as a valuable tool in advanced malware analysis and threat detection. It helps cybersecurity specialists by efficiently identifying important mechanisms and offering pertinent signs of compromise. Gemini Pro exhibits better coherence, clarity, and time efficiency than conventional techniques. These results demonstrate how the model's sophisticated natural language processing capabilities could improve malware analysis and mitigation.

By improving malware analysis with precise, quick interpretation of dangerous code, Gemini Pro has the potential to completely transform cybersecurity, according to the study. It can speed up threat detection and response by identifying important elements and signs of compromise. It facilitates communication between technical and non-technical parties by generating concise, comprehensible descriptions. By incorporating these NLP models into cybersecurity processes, overall security is strengthened, and threat response is enhanced [14].

Additional work might entail adjusting the model for tasks unique to cybersecurity and combining it with additional analytical tools. To guarantee the model's responsible and efficient application in changing cybersecurity environments, it will also be essential to investigate real-time threat detection, realistic deployment, and ethical issues.

Walton et al. proposed MalParse [15], malware semantic analyzer based on GPT-4o mini to enhance static reverse engineering of Android malware by leveraging Large Language Models (LLMs). Traditional malware analysis is often time-consuming and requires expert knowledge, particularly when dealing with advanced, obfuscated, or zero-day malware samples. MalParse addresses these challenges by automating the reverse engineering process and using a tier-based summarization method that provides detailed summaries at the function, class, and package levels. This structured approach enables security analysts to identify the root causes of malicious behavior within an application more efficiently. The system comprises of three key components: auto-decompilation and feature extraction using tools like Dex2Jar and CFR, hierarchical code summarization compatible with GPT-4o-mini, and advanced prompt engineering to guide the LLM in malware detection tasks. In this research, three types of prompts were used—Vanilla (with no malware-specific context), API-Scoped (highlighting sensitive APIs and permissions), and Malware-Scoped (focused on behaviors indicative of malware). Among these, Malware-Scoped prompting achieved the highest accuracy, correctly classifying 77% of the 200 Android applications tested, outperforming the Vanilla (49.5%) and API-Scoped (56%) prompts [15].

A notable feature of MalParse is its ability to generate "chain-of-thought" summaries. These summaries provide analysts with a transparent decision-making trail, linking high-

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

level behaviors to specific code structures. For instance, one case study accurately identified malware involving privilege escalation and dynamic code execution by analyzing specific class methods and their use of Java reflection and class loaders. Despite its strengths, MalParse has some limitations. Its accuracy depends on the quality of the prompts and the capabilities of the LLM. Moreover, it is less effective against heavily obfuscated code and relies on accurate decompilation, which may not always succeed. To overcome these issues, future work should focus on improving the prompt process and accuracy, using locally built lightweight LLM, and initially analyzing bytecode rather than decompiling Java. Overall, MalParse represents a significant advancement in automated malware analysis by combining LLM-based summarization with automated reverse engineering and semantic analysis.

Otoum et al. [16] showcase in their research a unique methodology for encompassing LLMs into IoT cybersecurity. The proposed framework assists in merging real-time threat detection with automated prevention controls and mechanisms. This study also highlights the security limitations encountered in the traditional IoT ecosystem and how they can be mitigated by using LLM models such as BERT. To test this, the authors used multiple BERT versions such as BERT-Small and BERT-Mini and fine-tuned them so that they can be assessed on IoT-specific datasets (IoT-23 and TON IoT). The results of this experiment were promising because: (1) the test did not overload the systems, showcasing its efficiency while being run in a resource-constrained environment; (2) the experiment recorded a detection accuracy score of 99.75%.

This study developed a dual-component system combining an LLM-based detection engine with an automated prevention module using decision-tree logic for actions like IP blocking and traffic redirection—operating without human input. Deployed via Docker containers, the framework was tested across multiple edge-cloud environments, demonstrating improved detection accuracy, lower latency, and better resource efficiency compared to traditional ML and rule-based systems. However, the isolated Docker setting limits real-world applicability, particularly for diverse IoT environments and broader attack types beyond DDoS. The study also lacks precise estimates of real-world computational demands. Future work should test the system in actual IoT deployments, expand its threat coverage beyond DDoS, and integrate approaches like federated learning and explainable AI to enhance threat intelligence, preserve privacy, and improve analyst interpretability.

TABLE I
OVERVIEW OF THE PAPERS REVIEWED IN OUR STUDY

Authors	Main Objective	Contribution/findings
Mogage A. (2024) [10]	Rule-based binary analysis framework with LLM to assist human analysts in defining behavioural rules that identify specific malware.	Hybrid Framework, LLM-guided precise rule creation by suggesting relevant windows APIs, common attack, C++ examples etc
Feng et al. (2025) [5]	LLM-MalDetect for Android malware detection	Treat sequences of Android API calls as language to better understanding malware behaviour semantic
Wong M. et al. (2025) [11]	Evaluate GPT-4o can assist malware analyst in identifying and bypassing evasive techniques	GPT-4o outperformed symbolic execution, successfully analysed real malware samples evaded detection
Zhao et al. (2025) [12]	AppPoet, LLM-assisted system for detecting Android malware and generating diagnostic report.	Multi-view prompt engineering approach to enhance quality and stability of LLM output.
Jelodar et al. (2025) [17]	Review of LLM-based approaches in malware code analysis	Prompt tuning, few-shot learning enhances LLM performance, dual use of LLM in explaining insecure code and generating secure alternatives
Williamson et al. (2024) [14]	LLMs can assist malware reverse engineering by interpreting disassembled code	GPT-4 can infer accurate function names from stripped binaries, recover semantic meaning from disassembled code.
Walton et al. (2024) [15]	MalParse – automate and enhance static reverse engineering of Android malware	GPT-4o-mini – semantic analysis framework, structured code summarization
Sanchez et al. (2024) [13]	Develop LLM-based framework for malware detection in high-secure environments by analysing system call behaviour at OS level	Combines LLMs with syscall analysis for detecting sophisticated and zero-day malware
Otoum et al. (2025) [16]	LLM-based framework for real-time malware detection and autonomous prevention in IoT systems	Fine-tuned LLM module + rule-based decision tree prevention module, BERT Small with high accuracy (Docker, edge-cloud)

V. DISCUSSION

The findings of this systematic review indicate that the integration of Large Language Models (LLMs) into malware detection and analysis is an emerging yet rapidly evolving field with promising results. Our review reveals a growing consensus that LLMs, when used for semantic feature extraction, behavioral log interpretation, or zero-shot classification, offer a significant advancement over traditional static and dynamic malware analysis techniques.

In response to RQ1, the reviewed studies demonstrate diverse methods for integrating LLMs into security workflows, including prompt engineering, transformer-based embeddings for API sequences (e.g., LLM-MalDetect), and hybrid systems combining LLMs with SVMs or Random Forest classifiers. These architectures improve detection of polymorphic and previously unseen malware, a limitation long faced by signature-based tools.

Addressing RQ2, the literature highlights that LLMs excel in contextualizing obfuscated code and identifying latent intent behind code execution paths, especially when integrated with threat intelligence and runtime data. However, key limitations persist, including high computational demands, explainability challenges, token length constraints, and the reliance on

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.

curated or synthetic datasets. Some studies also report that LLMs can inadvertently generate malicious code, underscoring ethical concerns around dual use.

For RQ3, the future of LLM-assisted malware detection lies in expanding real-world dataset availability, developing lightweight and federated LLM architectures, and integrating Explainable AI (XAI) techniques to enhance trust and interpretability. Furthermore, robust watermarking and adversarial robustness testing are critical for ensuring responsible and secure deployment. As LLMs increasingly intersect with malware research, they must be framed not just as tools for detection, but as platforms for proactive, scalable, and adaptive cybersecurity defense.

VI. Conclusion and Future Work

This review shows that the emergence of Large Language Models can be utilized to assist with the process of malware detection. Moreover, throughout the reviewed studies, it was observed that LLMs are surpassing traditional systems and older machine learning models in their ability to detect malware, obfuscation, and zero-day threats and vulnerabilities. In addition, it was also apparent that LLMs are diverse in their capabilities, ranging from being able to handle semantic reasoning and zero-shot classification to reverse engineering malware behaviour based on static and dynamic features.

This study also investigates the implementation of LLM through tools such as LLM-MalDetect, AppPoet, MalParse and Gemini Pro. Each of these works and functions in different systems and environments such as the Android and IoT ecosystems. Despite these advancements, the models in the reviewed studies all had their limitations such as LLM hallucinations, lack of standardization benchmarks, applicability to function in real/resource-constrained environments, and ethical oversight due to dual-use concerns. To compensate for these limitations, possible enhancements were also highlighted in this study such as investigating the possibility of integrating LLMs with SIEMs and EDRs, and creating models that are functional in a resource-constrained environment, to name a few. In summary, the results seen in the studies show that deploying LLM to perform malware detection can be beneficial to organizations but needs to also be done in a way that is secure and efficient.

Future research should prioritize making LLM-driven malware detection more transparent, efficient, and operationally viable. Efforts are needed to improve explainability and trust, develop lightweight models for edge and IoT environments, and establish standardized benchmarks for fair evaluation. Equally important is the integration of LLMs into real-world security operations to support detection, response, and analyst collaboration. Addressing ethical concerns, governance, and adversarial robustness will ensure that these technologies evolve responsibly while strengthening resilience against emerging threats.

References

[1] F. A. Aboaoja, A. Zainal, F. A. Ghaleb, B. A. S. Al-Rimy, T. A. E. Eisa, and A. A. H. Elnour, "Malware Detection Issues, Challenges, and Future Directions: A Survey," *Applied Sciences*, vol. 12, no. 17, p. 8482, 2022.

[2] N. Z. Gorment, A. Selamat, L. K. Cheng, and O. Krejcar, "Machine Learning Algorithm for Malware Detection: Taxonomy, Current Challenges, and Future Directions," *IEEE Access*, vol. 11, pp. 141 045–141 089, 2023.

[3] A. Cletus, A. A. Opoku, and B. A. Weyori, "An Evaluation of Current Malware Trends and Defense Techniques: A Scoping Review with Empirical Case Studies," *Journal of Advances in Information Technology*, vol. 15, no. 5, 2024.

[4] M. G. Gaber, M. Ahmed, and H. Janicke, "Malware Detection with Artificial Intelligence: A Systematic Literature Review," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–33, 2024.

[5] R. Feng, H. Chen, S. Wang, M. M. Karim, and Q. Jiang, "LLM-MalDetect: A Large Language Model-Based Method for Android Malware Detection," *IEEE Access*, 2025.

[6] Y. Zhang, X. Zhou, H. Wen, W. Niu, J. Liu, H. Wang, and Q. Li, "Tactics, Techniques, and Procedures (TTPs) in Interpreted Malware: A Zero-Shot Generation with Large Language Models," *arXiv preprint arXiv:2407.08532*, 2024.

[7] A. Kempen, "WormGPT/FraudGPT: The Ugly, Dangerous Cousin of ChatGPT," *Servamus Community-Based Safety and Security Magazine*, vol. 117, no. 11, pp. 19–21, 2024.

[8] J. Al-Karaki, M. A. Z. Khan, and M. Omar, "Exploring LLMs for Malware Detection: Review, Framework Design, and Countermeasure Approaches," *arXiv preprint arXiv:2409.07587*, 2024.

[9] AV-Atlas, "New Malware 2025 Statistics," https://portal.av-atlas.org/malware, 2025.

[10] A. Mogage, "AI-Assisted Malware Capabilities Capturing," *Procedia Computer Science*, vol. 246, pp. 860–869, 2024.

[11] M. Y. Wong, K. Valakuzhy, M. Ahamad, D. Blough, and F. Monrose, "Understanding LLMs' Ability to Aid Malware Analysts in Bypassing Evasion Techniques," in *Companion Proceedings of the 26th International Conference on Multimodal Interaction*, November 2024, pp. 36–40.

[12] W. Zhao, J. Wu, and Z. Meng, "AppPoet: Large Language Model-Based Android Malware Detection via Multi-View Prompt Engineering," *Expert Systems with Applications*, 2025.

[13] P. M. Sánchez, A. H. Celdrán, G. Bovet, and G. M. Pérez, "Transfer Learning in Pre-Trained Large Language Models for Malware Detection Based on System Calls," in *MILCOM 2024 - IEEE Military Communications Conference*, Washington, DC, USA, 2024, pp. 853–858.

[14] A. Q. Williamson and M. Beauparlant, "Malware Reverse Engineering with Large Language Models for Superior Code Comprehensibility and IoC Recommendations," *Lumina Cognition Lab*, 2024.

[15] B. J. Walton, M. E. Khatun, J. M. Ghawaly, and A. Ali-Gombe, "Exploring Large Language Models for Semantic Analysis and Categorization of Android Malware," in *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, Honolulu, HI, USA, 2024, pp. 248–254.

[16] Y. Otoum, A. Asad, and A. Nayak, "LLM-Based Threat Detection and Prevention Framework for IoT Ecosystems," *arXiv preprint arXiv:2505.00240*, 2025.

[17] H. Jelodar *et al.*, "Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering," *arXiv preprint arXiv:2504.07137*, 2025.

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 15:25:31 UTC from IEEE Xplore. Restrictions apply.